

## 二维数组的"方寸之间" (17分)

```
1  #include <stdio.h>
2  int main() {
3      int a[2][3] = {{1,2,3},
4                      {4,5,6}};
5      int *p = a[0];
6      int (*q)[3] = a;
7
8      printf("%d\n", *(p + 3) + (*(q + 1) + 1));
9      printf("%d\n", *(a[1] - 1) + *(a[0] + 4));
10     printf("%zu\n", sizeof(a + 1) + sizeof(*a));
11     printf("%d\n", (p + 2)[1] - (*(q + 1))[-1]);
12     return 0;
13 }
```

### 问题

#### 1. 写出程序输出结果 (4分)

#### 2. 逐个表达式分析

- `*(p + 3)` 访问的是哪个元素? 为什么? (2分)
  - 访问的是 `a[1][0]` (即4)
  - 原因: `p` 指向 `a[0][0]`, 指针+1就移动一个 `int` 大小, +3偏移到第4个元素, 在连续存储中即第1行第0列。
- `*(a[1] - 1)` 的值是多少?数组在内存中存储有什么特点? (3分)
  - `a[1]` 指向元素 4, 减1后指向前一元素 3, 因为二维数组在内存中是连续线性存储的
- `sizeof(a + 1)` 的结果是多少? 如果是 `sizeof(a)` 值又是多少? 说明原因 (2分)
  - 在表达式中, `a` 退化为指针, `a+1` 是指针类型 (`int (*)[3]`), 其大小固定为指针大小 (64位系统是 8, 32位系统是 4)
  - `sizeof(a)` 结果: 24 (`2 * 3 * sizeof(int) = 2 * 3 * 4 = 24`)  
在 `sizeof(a)` 中: `a` 是数组名, 但 `sizeof` 是特殊情况, 不会发生退化, `sizeof` 直接计算整个数组所占的字节数, 数组 `a[2][3]` 有 2 行, 每行 3 个 `int`, 共 6 个 `int`, 每个 `int` 占 4 字节, 结果为 24
- `(p + 2)[1]` 等价于什么形式的指针运算? (1分)
  - 等价于 `*(p + 2 + 1)`, 即 `*(p + 3)`
- `*(q + 1)[-1]` 中的 `[-1]` 访问的是哪个元素? 并说明原因 (2分)
  - 访问的是 `a[0][2]`, 即 3
  - 因为 `*(q+1)` 是第1行首地址, `[-1]` 表示向前一个元素, 即第0行最后一个元素。

### 3. 概念辨析

- `p` 和 `q` 的类型分别是什么？它们的步长各是多少？ (2分)
  - `p` 类型：数组指针, `int *`, 步长 = `sizeof(int)` = 4字节
  - `q` 类型：指针数组, `int (*)[3]` (指向含3个int的数组的指针), 步长 = `3 * sizeof(int)` = 12字节
- 如果数组改为 `int a[2][3] = {0};`, `sizeof(a+1) + sizeof(*a)` 的结果会变吗？为什么？ (2分)
  - 不会变
  - 因为 `sizeof` 在编译期求值, 与数组内容无关;
    - `sizeof(a+1)` 仍是指针大小 (8)
    - `sizeof(*a)` 仍是第一行大小 (12)

## 题解

这里要说出三个输出中详细的计算过程

分步推导：

```
1 // 第1行
2 *(p + 3) = 4 // p指向1, +3跳到4 (跨行了)
3 (*(q + 1) + 1) = 5 // q+1指向第1行, 解引用得{4,5,6}首地址, +1取5
4 结果 = 4 + 5 = 9
5
6 // 第2行
7 *(a[1] - 1) = 3 // a[1]指向4, -1指向3 (跨行往前)
8 *(a[0] + 4) = 5 // a[0]指向1, +4跳到5
9 结果 = 3 + 5 = 8
10
11 // 第3行
12 sizeof(a + 1) = 8 // a+1退化为指针
13 sizeof(*a) = 12 // *a是a[0], 3个int = 12字节
14 结果 = 8 + 12 = 20
15
16 // 第4行
17 (p + 2)[1] = 6 // p 是指针数组, p表示的是第一行的第一个元素 1
18 // (p + 2)[1]是p 先后移动 3 个int, 指向 4
19
20
21
22 (*(q + 1))[-1] = 3 // q+1指向第1行{4,5,6}, 解引用后指向4, [-1]取3 // q是数组指针, 指向
  第一个数组第一个元素
23 // (q + 1)指向第二个数组, (q + 1)[-1]指的是向前移动一位, 指向 3
24 结果 = 4 - 3 = 1
```

验证：

```

1 printf("%d\n", *(p + 3) + (*(q + 1) + 1)); // 4 + 5 = 9
2 printf("%d\n", *(a[1] - 1) + *(a[0] + 4)); // 3 + 5 = 8
3 printf("%zu\n", sizeof(a + 1) + sizeof(*a)); // 8 + 12 = 20
4 printf("%d\n", (p + 2)[1] - (*(q + 1))[-1]); // 4 - 3 = 1

```

## 字符数组的"三重门" (16分)

```

1 #include <stdio.h>
2 #include <string.h>
3
4 void transform(char *s) {
5     char *p = s + strlen(s) - 1;
6     while (p > s) {
7         *p = *(p - 1);
8         p--;
9     }
10    *s = 'X';
11 }
12
13 int main() {
14     char str[20] = "3G";
15     char *ptr = str + 1;
16
17     strcpy(str + 2, "lab");
18     transform(str);
19     strcat(ptr, "2025");
20
21     printf("%d\n", strlen(str) - sizeof(str) / sizeof(char));
22     printf("%c\n", *(str + 3));
23     printf("%s\n", ptr + 1);
24
25     return 0;
26 }

```

## 问题

1. 写出3个printf的输出结果 (3分)

```

1 -11
2 1
3 Gla2025

```

2. 初始 `str` 和 `ptr` 分别指向什么? (1分)

```
1 char str[20] = "3G";    // str = "3G\0"
2 char *ptr = str + 1;    // ptr 指向 str[1], 即 'G'
```

3. `strcpy(str + 2, "lab")` 后 `str` 变成什么? (1分)

`str` 变为 "3Glab" (长度为5)

4. `transform(str)` 函数做了什么? (2分)

```
1 void transform(char *s) {
2     char *p = s + strlen(s) - 1; // p指向最后一个字符
3     while (p > s) {
4         *p = *(p - 1); // 将前一个字符复制到当前位置
5         p--;
6     }
7     *s = 'X';
8 }
```

这个函数的作用: 所有字符向右移动一位, 第一个位置放'X'

5. `transform` 中的 `while (p > s)` 条件如果改成 `while (p >= s)` 会怎样? (1分)

- 最后一次循环 `p == s` 时, 会执行 `*p = *(p-1)`, 但 `p-1` 指向 `s-1` (数组前一个位置), 这是非法访问, 会导致未定义行为 (可能崩溃或产生错误结果)

6. `transform` 执行后, `ptr` 的指向变了吗? 为什么? (2分)

- `ptr` 的指向没有变!
  - `ptr` 存储的是地址值, `transform` 函数只修改了数组内容, `ptr` 仍然指向 `str[1]` (索引1的位置), 但该位置的内容从原来的 'G' 变成了 '3'

7. `strcat(ptr, "2025")` 从哪里开始追加? 会覆盖什么? (2分)

- `strcat(ptr, "2025")` 从 `ptr` 指向的字符串末尾开始追加:
- 会覆盖掉末尾的 `'\0'`

8. ◦ 最终 `str` 和 `ptr` 的内容分别是什么? (2分)

- `str` 的内容: "X3Gla2025" (整个数组)
- `ptr` 指向的内容: "3Gla2025" (从索引1开始)

9. ◦ 请说明 `sizeof(str) / sizeof(char)` 的值, 以及计算的过程 (2分)

- `str` 被声明为长度为 20 的 `char` 数组, `sizeof(str)` 计算的是整个数组占用的字节数
- `char` 类型占 1 个字节, 所以 `sizeof(char) = 1`

- 因此: `sizeof(str) = 20 * 1 = 20` 字节

## 题解

初始时候, `str` 与 `ptr`

```
1 char str[20] = "3G";    // str指向数组首地址, 内容为"3G\0"  
2 char *ptr = str + 1;    // ptr指向str[1], 即字符'G'
```

内存布局:

```
1 索引:  0      1      2      3      4      5 ...  
2      +---+---+---+---+---+---+  
3 str:  | '3' | 'G' | '\0' | ?   | ?   | ?   |  
4      +---+---+---+---+---+---+  
5          ↑   ↑  
6        str ptr
```

```
1 strcpy(str + 2, "lab");
```

`str = "3Glab"` (长度为5)

```
1 strcat(ptr, "2025");
```

`strcat(ptr, "2025")` 会在 `ptr` 指向的字符串末尾 (即索引5的'\0'处) 追加 "2025"

此时整个 `str = "X3Gla2025"` (长度为9)

但注意:

- `ptr` 指向的字符串是 `"3Gla2025"` (从索引1开始)
- 整个数组内容是 `"X3Gla2025"`

## 第1个 printf

```
1 printf("%d\n", strlen(str) - sizeof(str) / sizeof(char));
```

计算:

- `strlen(str)` = 9 ("X3Gla2025" 的字符数)
- `sizeof(str)` = 20 (整个数组大小, 单位: 字节)
- `sizeof(char)` = 1
- `sizeof(str) / sizeof(char)` = 20 / 1 = 20
- 结果: **9 - 20 = -11**

输出: `-11`

---

## 第2个 printf

```
1 printf("%c\n", *(str + 3));
```

- `str + 3` 指向索引3
- 索引3的字符是 'l' (从上面的内存图可以看出)

输出: `l`

---

## 第3个 printf

```
1 printf("%s\n", ptr + 1);
```

- `ptr` 指向索引1 ('3')
- `ptr + 1` 指向索引2 ('G')
- 从索引2开始到'\0'的字符串是 `"Gla2025"`

输出: `Gla2025`